实验 13　虚拟化

13.1 实验目标

（1）了解 openEuler 常见的虚拟化方法

（2）基于 openEuler 部署 QEMU 和 Docker

13.2 预备知识

13.2.1 QEMU

QEMU 是一个模拟器，它向 Guest OS 模拟 CPU 和其他硬件，Guest OS 认为自己和硬件直接打交道，其实是同 QEMU 模拟出来的硬件打交道，Qemu 将这些指令转译给真正的硬件。由于所有的指令都要从 QEMU 里面过一手，因而性能较差。

QEMU 作为系统模拟器，可以模拟出一台能够独立运行操作系统的虚拟机。如下图所示，每个虚拟机对应主机(Host)中的一个 QEMU 进程，而虚拟机的 vCPU 对应 QEMU 进程的一个线程。

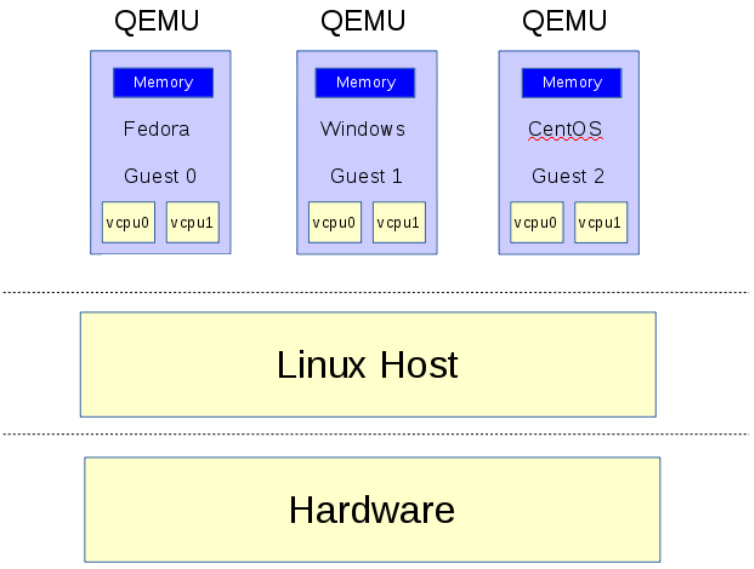


图 13.1　QEMU

13.2.2 KVM

KVM 是 Linux 的内核模块，它需要 CPU 的支持，采用硬件辅助虚拟化技术 Intel-VT，AMD-V。KVM 通过/dev/kvm 暴露接口，可通过`ls /dev/kvm`或 `ls

/sys/module/kvm` 查看是否存在相应目录/文件，来确定内核是否支持 KVM 虚拟化。若上述文件/目录不存在，则说明系统内核编译时未开启 KVM 虚拟化，需要更换支持 KVM 虚拟化的 Linux 内核。

KVM 内核模块本身只能提供 CPU 和内存的虚拟化，所以它必须结合 QEMU 才能构成一个完成的虚拟化技术，即 qemu-kvm。

qemu-kvm：QEMU 将 KVM 整合进来，通过 ioctl 调用/dev/kvm 接口，将有关 CPU 指令的部分交由内核模块来做。KVM 负责 CPU 虚拟化+内存虚拟化，实现了 CPU 和内存的虚拟化，但 KVM 不能模拟其他设备；而 QEMU 模拟 IO 设备（网卡，磁盘等），KVM 加上 QEMU 之后就能实现真正意义上服务器虚拟化。因为用到了上面两个东西，所以称之为 qemu-kvm。

13.2.3 libvirt

libvirt 是目前使用最为广泛的对 KVM 虚拟机进行管理的工具和 API。libvirtd 是其 daemon 进程，可以被本地的 virsh 调用，也可以被远程的 virsh 调用，libvirtd 调用 qemu-kvm 操作虚拟机。

libvirt 工具采用 XML 格式的文件描述一个虚拟机特征，包括虚拟机名称、CPU、内存、磁盘、网卡、鼠标、键盘等信息。用户可以通过修改配置文件，对虚拟机进行管理。

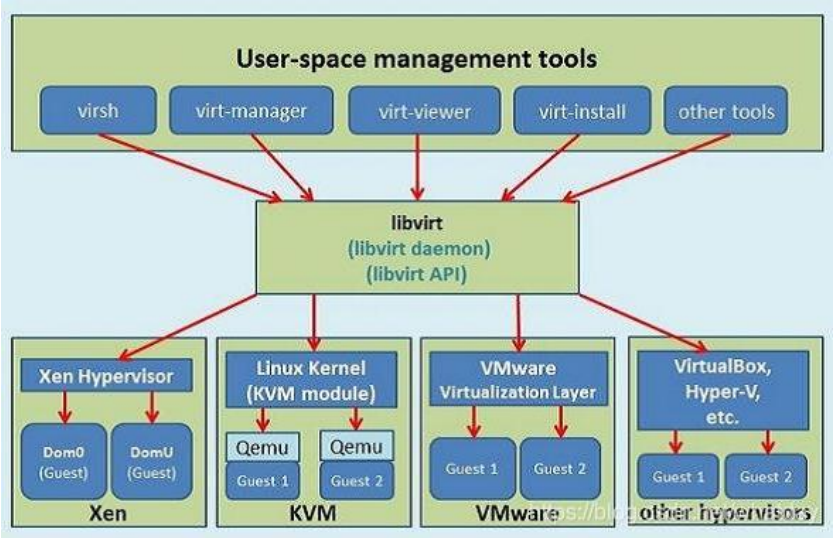


图 13.2　libvirt

13.2.4 Docker 简介

操作系统中创建虚拟机存在资源占用多、冗余步骤多、启动慢等缺点，Linux

发展出了另一种虚拟化技术：Linux 容器（LXC，Linux Containers）。Linux 容器不是模拟一个完整的操作系统，而是通过 cgroup 与系统其他部分隔离开的一系列进程。Linux 容器具有极佳的可移植性，但前提是它们必须与底层系统兼容。

Docker 属于 Linux 容器的一种封装，提供简单易用的容器使用接口。用户操作 Docker 容器就像操作一个快速轻量级的虚拟机一样简单。它是目前最流行的 Linux 容器解决方案。相比虚拟机，具有：启动快、体积小、资源占用少等优势。

(1)Docker Engine

Docker Engine 属于 Docker 的运行层。这是一套轻量化运行时及工具组合，负责管理容器、镜像、构建等。它以原生方式运行在 Linux 系统之上，并由以下元素构成：

①Docker Daemon，守护程序进程，运行在主机计算机之上。用于侦听 Docker API 请求并管理 Docker 对象，例如镜像，容器，网络和卷；还可以与其他守护程序通信以管理 Docker 服务。

②Docker Client CLI，负责与 Docker Daemon 通信以执行命令。Docker Client 是我们作为最终用户的通信对象，可将其视为 Docker 的 UI。我们进行的一切操作都将直接接入 Docker Client，再由其将指令传递至 Docker Daemon。例如：当使用 docker run 命令时，客户端该命令发送到 dockerd，以执行这些命令。

③REST API，用于同 Docker Daemon 远程交互。

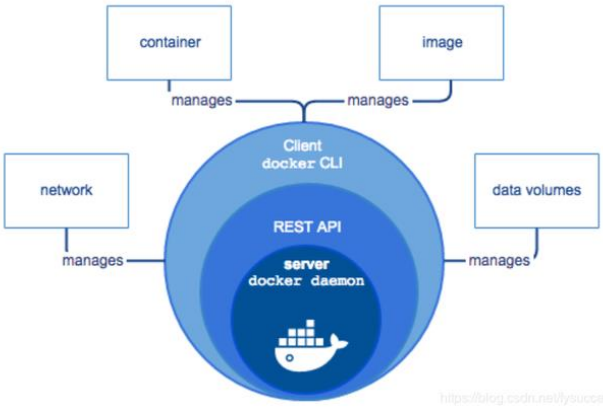


图 13.3　Docker Engine

Docker 是 C/S 架构的程序，Docker 的 client 向守护进程 daemon 发起请求，守护进程处理完成后返回结果。Docker client 既可以在本地访问守护进程，也可以远程访问守护进程。

(2)Docker 对象

Docker 管理的对象包含镜像、容器、网络、数据卷等。

①镜像

Docker 镜像是一个特殊的文件系统，提供了运行时需要的程序、库、资源、配置等文件，还包含一些为运行时准备的配置参数（如环境变量、用户等），镜像不包含任何动态数据，其内容在构建之后也不会被改变。

镜像的文件系统有一个分层存储的概念，采用的是 Union FS 技术，因此，镜像并不是简单地由一组文件组成，而是由多层文件系统叠加联合组成。如下图所示：

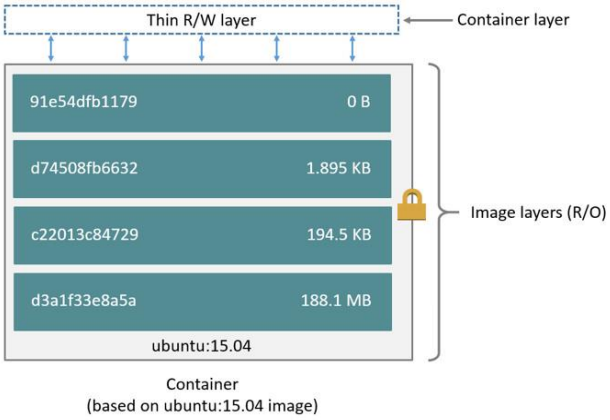


图 13.4　Docker 镜像

镜像构建时会一层一层地构建，前一层是后一层基础，每层构建完后就变成只读的，不会再发生改变。镜像分层存储的一大好处是复用，镜像的每一层可以在不同镜像间复用，这就好比我们开发项目时将一些公共功能封装成 jar 包，在各个项目可以直接依赖使用一样。

②容器

相对镜像，容器是一个动态的运行时的概念，它与镜像的关系类似于面向对象中类与实例的关系。

容器可以被创建、启动、停止、删除等。容器运行实质上就是运行一个进程，但与那些直接在宿主机上运行的进程不同，容器运行在自己的独立的隔离的命名空间中——拥有自己的 root 文件系统、网络配置、进程空间，甚至自己的用户 ID 空间，因此虽然是以进程的形式运行，但好像是运行在一个独立的系统中一样，这样相比直接运行于宿主机的进程，容器的运行显得更为安全。

前面说到镜像的分层存储概念，对于容器来说，实际上也是以镜像作为基础层，在其上创建了一个当前容器的存储层，如下图：

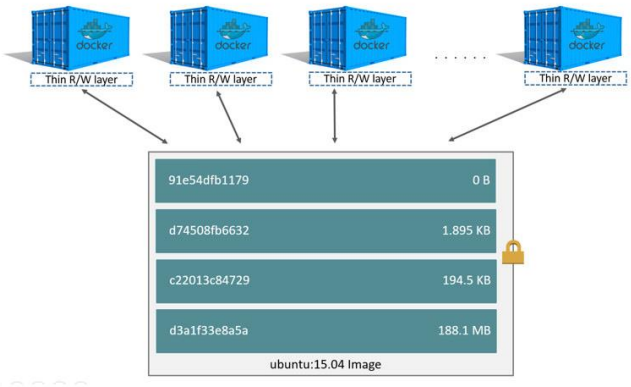


图 13.5　Docker 镜像

以镜像 ubuntu 15.04 为基础层所创建的容器，都有一个自己的可读写的存储层（镜像的存储层是只读的）。容器存储层的生命周期与容器一样，容器销毁时，容器的存储层也会随之消亡，任何保存在容器存储层的数据也都会随容器的删除而丢失，因此一般我们要保持容器存储层的无状态化，所有文件的写操作，都应该使用数据卷或绑定宿主机目录。

③数据卷

数据卷是一个独立于容器，可供一个或多个容器使用的特殊目录，它绕过了 Union FS，不会随容器的销毁而消亡。数据卷的使用，类似于 Linux 下对目录或文件进行 mount。

数据卷具备如下特性：

①可以在容器之间共享和重用。

②对数据卷的修改会立马生效。

③数据卷的更新，不会影响到镜像。

④数据卷默认会一直存在，不会随容器的删除而消亡。

13.3 实验步骤

13.3.1 任务 1

使用 libvirt+xml 配置文件的方法，搭建 QEMU 虚拟机运行环境。

(1)安装虚拟化组件：

```
[root@localhost ~]# dnf install -y libvirt qemu
```

(2) 验证安装是否成功:

①首先查看内核是否支持 KVM 虚拟化，即验证/dev/kvm 和/sys/module/kvm 文件是否存在。注意，在这一步中需要将内核切换成 openeuler 原本的内核：

```
[root@localhost ~]# ls /dev/kvm
/dev/kvm
[root@localhost ~]# ls /sys/module/kvm
parameters    uevent
```

②确认 QEMU 是否安装成功。若安装成功则可以看到 QEMU 软件包信息：

```
[root@localhost ~]# rqm -qi qemu
```

③确认 libvirt 是否安装成功:

```
[root@localhost ~]# rpm -qi libvirt
```

(3)启动 libvirtd 服务：

```
[root@localhost ~]# systemctl start libvirtd
```

(4)编写虚拟机 xml 配置文件：

x86_64 架构的 XML 配置文件：

```
<domain type='kvm'>
    <name>openEulerVM</name>
    <memory unit='GiB'>8</memory>
    <vcpu>4</vcpu>
    <os>
    <type arch='aarch64' machine='virt'>hvm</type>
    <loader readonly='yes' type='pflash'>/usr/share/edk2/aarch64/QEMU_EFI-pflash.raw</loader>
    <nvram>/var/lib/libvirt/qemu/nvram/openEulerVM.fd</nvram>
    </os>
    <features>
    <acpi/>
    <gic version='3'/>
    </features>
    <cpu mode='host-passthrough'>
        <topology sockets='2' cores='2' threads='1'/>
    </cpu>
    <iothreads>1</iothreads>
    <clock offset='utc'/>
    <on_poweroff>destroy</on_poweroff>
    <on_reboot>restart</on_reboot>
    <on_crash>restart</on_crash>
    <devices>
    <emulator>/usr/libexec/qemu-kvm</emulator>
```

```
        <disk type='file' device='disk'>
            <driver name='qemu' type='qcow2' iothread="1"/>
            <source file='/mnt/openEuler-image.qcow2'/>
            <target dev='vda' bus='virtio'/>
            <boot order='1'/>
        </disk>
        <disk type='file' device='cdrom'>
            <driver name='qemu' type='raw'/>
            <source file='/mnt/openEuler-20.03-LTS-aarch64-dvd.iso'/>
            <readonly/>
            <target dev='sdb' bus='scsi'/>
            <boot order='2'/>
        </disk>
        <interface type='bridge'>
            <source bridge='br0'/>
            <model type='virtio'/>
        </interface>
        <console type='pty'/>
            <video>
                <model type='virtio'/>
            </video>
            <controller type='scsi' index='0' model='virtio-scsi'/>
        <controller type='usb' model='ehci'/>
        <input type='tablet' bus='usb'/>
        <input type='keyboard' bus='usb'/>
        <graphics type='vnc' listen='0.0.0.0' passwd='n8VfjbFK'/>
        </devices>
        <seclabel type='dynamic' model='dac' relabel='yes'/>
    </domain>
```

(5)可以使用以下 virsh 命令操作虚拟机：

```
[root@localhost ~]# virsh define vm_openEuler.xml     #创建虚拟机
[root@localhost ~]# virsh edit openEulerVM            #定义/创建虚拟机后，再次修改 xml 文件的配置内容
[root@localhost ~]# virsh start openEulerVM           #启动虚拟机
[root@localhost ~]# virsh destroy openEulerVM         #关闭虚拟机
[root@localhost ~]# virsh undefined openEulerVM       #取消定义/删除虚拟机
```

13.3.2 任务 2

安装并使用 docker，完成 docker 容器的新建、启动、守护态运行、终止与删除等操作。

(1)输入以下命令以安装 docker：

```
[root@localhost ~]# dnf -y install docker
```

(2)启动 docker：

启动容器有两种方式：

①一种是基于镜像新建一个容器并启动；

②另外一个是将在终止状态（stopped）的容器重新启动。

例如，下面的命令输出一个 “Hello World”，之后终止容器：

```
[root@localhost ~]# sudo docker run ubuntu:14.04 /bin/echo 'Hello world'
```

这跟在本地直接执行 /bin/echo 'hello world' 几乎感觉不出任何区别。

下面的命令则启动一个 bash 终端，允许用户进行交互；在交互模式下，用户可以通过所创建的终端来输入命令；输入 exit 可推出交互模式。

```
[root@localhost ~]# sudo docker run -it ubuntu:14.04 /bin/bash
```

说明：

-t 选项让 Docker 分配一个伪终端（pseudo-tty）并绑定到容器的标准输入上；

-i 则让容器的标准输入保持打开。

(3)守护态运行

使用-d 参数可以让 Docker 容器在后台以守护态形式运行：

```
[root@localhost ~]# sudo docker run -d ubuntu:14.04 /bin/sh -c “while true; do echo hello world; sleep 1; done”
```

(4) 使用 docker stop container_id 来终止一个运行中的容器。使用"docker ps -n 2"查看最后被创建的 2 个 container_id。

```
[root@localhost ~]# docker ps -n 2；
[root@localhost ~]# docker stop container_id；
```

(5)删除容器

使用 docker rm container_id 来删除一个处于终止状态的容器。

```
[root@localhost ~]# docker rm container_id；
```

13.3.3 任务 3

编写 Dockerfile 文件，创建基于 Ubuntu 镜像，创建打印“Hello world”的 docker 镜像，并验证其可用性。

(1)新建空目录：

```
[root@localhost ~]# mkdir docker_test
```

(2)转到新目录：

```
[root@localhost ~]# cd docker_test/
```

(3)使用 vi 命令编辑 Dockerfile 文件：

```
[root@localhost ~]# vi Dockerfile
```

(4)按下 i 键进入 INSERT 模式并输入以下代码：

```
# This dockerfile uses the ubuntu image
# VERSION 2 - EDITION 1
# Author: docker_user
# Command format: Instruction [arguments / command] ..

# Base image to use, this must be set as the first line
FROM ubuntu

# Maintainer: docker_user <docker_user at email.com> (@docker_user)
MAINTAINER docker_user docker_user@email.com

# Commands to update the image
RUN echo "deb http://archive.ubuntu.com/ubuntu/ raring main universe" >> /etc/apt/sources.list

# Commands when creating a new container
CMD echo “Hello world!”
```

(5)创建镜像：

```
[root@localhost ~]# docker image build -t helloworld_ubuntu .
```

(6)镜像创建完成后，启动容器：

```
[root@localhost ~]# sudo docker run --name test helloworld_ubuntu:latest
```

(7)出现”Hello World！”则实验运行成功。

13.4 实验讨论

(1)第一个任务中，使用 libvirt+xml 配置文件的方法来搭建 QEMU 虚拟机运行环境，你遇到了哪些挑战和难点？

(2)第二个任务中，你在安装和使用 Docker 过程中遇到了哪些问题？你是如何解决这些问题的？

(3)第三个任务中，你使用 Dockerfile 创建了一个基于 Ubuntu 镜像的 Docker 镜像，并验证了其可用性。你在编写 Dockerfile 过程中遇到了哪些问题？如何优化你的 Dockerfile？